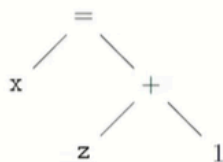


## 35. Графические и линейные IR, виды и применение.

### Синтаксические деревья и даги

Синтаксические деревья и даги относятся к высокоуровневым представлениям; описывают естественную иерархическую структуру исходной программы, хорошо подходят для статических проверок. Стандартный способ хранения синтаксического дерева — массив, элементами которого являются записи, соответствующие узлам дерева.



| Ind | Name | Data     |
|-----|------|----------|
| 1   | =    | 2 3      |
| 2   | x    | x.lexval |
| 3   | +    | 4 5      |
| 4   | z    | x.lexval |
| 5   | 1    | 1        |

Сигнатура внутреннего узла дерева выглядит как  $\langle op, l, r \rangle$ , где  $op$  — имя (метка) узла,  $l, r$  — номера значений левого и правого операнда соответственно. Чтобы построить даг вместо дерева, перед созданием нового узла выполняется поиск узла с заданной сигнатурой по массиву. При этом удобно использовать хэш, разбив весь массив на блоки примерно одинакового размера.

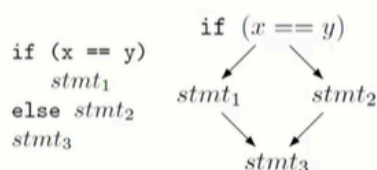
В проверке типов как раз мы строили синтаксические деревья и даги.

То, как организован массив, определяет, насколько быстро можно будет осуществлять операции на синтаксическом дереве, поиск узлов с заданными параметрами и непосредственные семантические проверки, для которых это IR (промежуточное представление) и предназначено.

Для построения синтаксического дага необходимо перед созданием нового узла проверить, не существует ли уже узел с заданной сигнатурой. Для этого все узлы разбиваются на группы с примерно равным кол-вом элементов и с помощью хэш-функции осуществляется доступ уже к конкретной группе узлов и среди них ищется заданный узел.

### Граф потока управления

Узлы графа потока управления (CFG) представляют из себя элементы компилируемой программы, рёбра — возможные передачи управления между ними.



Один из старейших видов IR; обычно используется в комбинации с другими: граф потока управления отражает взаимосвязи между блоками программы, в то время как инструкции внутри блока

представляются другим IR, например, синтаксическим деревом или каким-либо линейным представлением. Граф потока управления хорошо подходит для оптимизации генерируемого кода и отслеживания «мёртвого» кода.

Если узлом является какой-то блок, то внутри него не должно никуда передаваться управление.

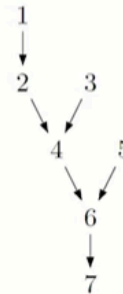
## Граф зависимости данных

Вспомогательный вид IR, отображающий поток данных между точками объявления и использования. Узлы графа соответствуют точкам объявлений и использований переменных в исходном коде. Дуга идёт от операции  $A$  к операции  $B$ , если операция  $B$  использует результаты  $A$ .

Рассмотрим пример инструкции присваивания  $w = w * 2 * x * y$  и соответствующего ей ассемблерного кода.

Данный вид IR используется для оптимизации. С его помощью обнаруживается возможный параллелизм, оптимизируется работа с памятью.

```
1. LOAD r1, @w
2. ADD r1,r1 ⇒ r1
3. LOAD r2, @x
4. MULT r1,r2 ⇒ r1
5. LOAD r2, @y
6. MULT r1,r2 ⇒ r1
7. STORE r1, @w
```



Отображает поток данных между всеми операциями, которые как-либо обращаются к переменным (объявление, использование, загрузка).

STORE - содержимое регистра, которое сейчас представляет из себя значение этого выражения, надо сохранить обратно в то место памяти, где хранится значение переменной  $w$ .

## Одноадресный код

Одноадресный код подразумевает наличие стека операндов.

Большинство операций работает со стеком: например, операция вычитания снимет два верхних элемента из стека и положит туда их разность. В наборе операций обычно есть `swap` — обмен двух верхних элементов стека.

Главное преимущество одноадресного кода в его компактности.

Стек создаёт неявное пространство имён, которое нет необходимости отображать в промежуточном представлении.

Одноадресный код легко генерировать и исполнять, и он является хорошим вариантом для сред, в которых есть жёсткие ограничения по памяти. Использовался для создания интерпретаторов байт-кода в Smalltalk, Java.

Все операции используют только один операнд, поэтому нужен стек.

Swap нужен для того, чтобы осуществлять некоммукативные операции.

## Двухадресный и трёхадресный коды

Команды двухадресного кода имеют вид  $x < op > y \Rightarrow x$  (один оператор и не более двух операндов). Инструкции двухадресного кода всегда переписывают один из операндов, и выбор правильного операнда является довольно трудоёмкой задачей компилятора. Представляет, скорее, исторический интерес.

Название «трёхадресный код» применяется ко многим различным IR, но во всех таких IR команда имеет вид  $x < op > y \Rightarrow z$  (один оператор и не более трёх операндов). Трёхадресные коды широко варьируются по списку операторов, уровню абстракции и способам хранения.

В двухадресном коде результат сохраняется в один из операндов - сложно выбрать именно тот, значение которого можно стереть (которое не понадобится в дальнейшем).

Трёхадресный код может быть ближе как к исходному коду, так и к целевому.

Во всех этих кодах сам адрес означает имя переменной, константа или временные имена, которые генерируются компилятором.

### Пример

Линейные представления выражения  $x - 2 * y$

| Одноадресный | Двухадресный                | Трёхадресный                    |
|--------------|-----------------------------|---------------------------------|
| push 2       | load 2 $\Rightarrow$ $t_1$  | load 2 $\Rightarrow$ $t_1$      |
| push y       | load y $\Rightarrow$ $t_2$  | load y $\Rightarrow$ $t_2$      |
| mult         | mult $t_2 \Rightarrow t_1$  | mult $t_1, t_2 \Rightarrow t_3$ |
| push x       | load x $\Rightarrow$ $t_3$  | load x $\Rightarrow$ $t_4$      |
| subtr        | subtr $t_1 \Rightarrow t_3$ | sub $t_4, t_3 \Rightarrow t_5$  |

$t_i$  — временные имена, генерируемые компилятором.

1. Операции над двумя верхними элементами стека (вытащили их, результат положили).
2. Временные имена это могут быть названия регистров, например. Переменная в правой части выступает в роли и операнда, и указания на место сохранения результата.
3. Для проверок может иметь одну структуру: более компактная, но в ней сложнее осуществлять перестановки. Для оптимизации кода: требуется больше памяти, но быстрее осуществляются перестановки.

## Статические единственные присваивания

IR, которое облегчает некоторые из оптимизаций кода. Все присваивания выполняются для переменных с различными именами.

| Трёхадресный          | СЕП                         |
|-----------------------|-----------------------------|
| $a + b \Rightarrow p$ | $a + b \Rightarrow p$       |
| $p - c \Rightarrow q$ | $p_1 - c \Rightarrow q_1$   |
| $q * d \Rightarrow p$ | $q_1 * d \Rightarrow p_2$   |
| $p + q \Rightarrow q$ | $p_2 + q_1 \Rightarrow q_2$ |

Одна и та же переменная может быть определена в двух разных путях потока управления:  
`if (flag) x = -1;  
else x = 1;`

`y = x * a;`

Если использовать различные имена для  $x$  в истинной и ложной части условной конструкции, то какое имя должно использоваться в `y = x * a`? СЕП использует соглашение о записи, называемое  $\phi$ -функцией:

```
if (flag)  $x_1 = -1$ ; else  $x_2 = 1$ ;  
 $x_3 = \phi(x_1, x_2) * a$ ;
```

Другая стратегия именования (по сравнению с трёхадресным кодом).

В СЕП нет переиспользования имен: с одной и той же переменной может что-то происходить в разных путях потока управления в программе, СЕП в каждом из этих путей присвоит переменной различные имена, а дальше с помощью соглашения о записи вернется то значение, которое реально сработало, и запишется снова в новую переменную.